

Fuzzing, CVEs & Reading Advisories

Fuzzing = machine-driven bug-hunt

```
Input far beyond valid -> feed the target millions of times
-> any crash/file/assert = a candidate bug
```

Types: dumb (random), grammar-aware, coverage-guided (the modern standard).

The coverage-guided loop (AFL++ / libFuzzer)

```
afl-fuzz -i in/ -o out/ ./target @@
# it feeds corpus, adds interesting samples, watches coverage, reports crashes
# read out/crashes/ - a crashing file = your proof
```

Triage like a pro

Reproduce the crash (run the crashing input again)

Note the exact fault: SEGV (offset?), heap corruption, assert

Classify: use-after-free, overflow, integer, format...

Minimize the input (afl-tmin / delta) → a small, clean repro

Decide impact without trembling: can you get R/W/E from it?

CVE literacy

```
CVE-2026-12345 (year + serial)
- The 'CVSS score' you must sanity-check (is it really 'critical'?)
- 'Affected versions' -> patch line -> your duty: update or mitigate
- KEV catalog (CISA): actively-exploited = PAUSE everything and patch
```

Reading advisories closely

The advisory tells you: affected product/ver, type (RCE/XSS/...),
workaround, CVE

Red-team reading: "this version is 2 years old → likely exploitable"

Blue-team reading: "patch within window or put in your risk register"

The ethics of 0days

Discovered: disclose responsibly (vendor timeline) or to a program

Never weaponize/harass; the '0day YouTuber' trend harms

Fuzzing finds in an afternoon what eyeballs in a month. But every crash is just a hypothesis until triaged – impact is what makes it a CVE.

[← Format String Attacks: Arbit...](#)

Module complete ✓